

Note 2: Project Object Model (POM) and Directory Structure

What is the POM?

The Project Object Model (POM) is the fundamental unit of work in Maven. It is an XML file named pom.xml that lives at the root of your project directory. The POM contains all the information Maven needs to build your project:

- Project identity (group, artifact name, version)
- Dependencies the project needs
- Plugins used during the build
- Build configuration and profiles
- Reference to a parent POM (if any)

Every Maven project must have a pom.xml. Maven reads this file and uses it to drive the entire build process.

Anatomy of a Basic pom.xml

```
<?xml version="1.0" encoding="UTF-8"?>

<project xmlns="http://maven.apache.org/POM/4.0.0"
  xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
  xsi:schemaLocation="http://maven.apache.org/POM/4.0.0
    http://maven.apache.org/xsd/maven-4.0.0.xsd">

  <modelVersion>4.0.0</modelVersion>

  <!-- Project Identity -->

  <groupId>com.example</groupId>

  <artifactId>my-app</artifactId>

  <version>1.0.0</version>

  <packaging>jar</packaging>

  <!-- Project Metadata -->
```

<name>My Application</name>

<description>A sample Maven project</description>

<!-- Java Version Configuration -->

<properties>

 <maven.compiler.source>17</maven.compiler.source>

 <maven.compiler.target>17</maven.compiler.target>

 <project.build.sourceEncoding>UTF-8</project.build.sourceEncoding>

</properties>

<!-- Dependencies -->

<dependencies>

 <dependency>

 <groupId>org.springframework.boot</groupId>

 <artifactId>spring-boot-starter-web</artifactId>

 <version>3.1.0</version>

 </dependency>

 <dependency>

 <groupId>junit</groupId>

 <artifactId>junit</artifactId>

 <version>4.13.2</version>

 <scope>test</scope>

 </dependency>

</dependencies>

</project>

Key POM Fields Explained

groupId A unique identifier for the organisation or group that created the project. It follows reverse domain name convention. Example: com.example, org.apache, io.github.username

artifactId The name of the project or module. This becomes the name of the output .jar or .war file. Example: my-app, user-service, payment-api

version The current version of the project. Maven uses semantic versioning — MAJOR.MINOR.PATCH.

- 1.0.0 — stable release
- 1.0.0-SNAPSHOT — a development version that is still being worked on; Maven always re-downloads SNAPSHOT versions to get the latest

packaging Defines the type of artifact Maven should produce:

- jar — default; a standard Java archive
- war — a web application archive for deployment to servlet containers like Tomcat
- pom — used for parent POMs that don't produce a build artifact themselves

properties A section for defining reusable values. Commonly used to set the Java version and encoding. Properties can be referenced elsewhere in the POM using \${property.name} syntax.

Maven Standard Directory Structure

Maven enforces a standard directory layout. Following this convention means Maven knows exactly where to find your source files, test files, and resources without any additional configuration.

my-app/

└─ pom.xml

└─ src/

| └─ main/

| | └─ java/

| | | └─ com/example/myapp/

| | | └─ App.java

| | └─ resources/

```
| |   └─ application.properties
|   └─ test/
|       └─ java/
|           └─ com/example/myapp/
|               └─ AppTest.java
|           └─ resources/
└─ target/
    └─ classes/
    └─ test-classes/
    └─ my-app-1.0.0.jar
```

Directory Breakdown:

Directory	Purpose
-----------	---------

src/main/java/	Application source code (.java files)
----------------	---------------------------------------

src/main/resources/	Non-Java resources (config files, properties, XML)
---------------------	--

src/test/java/	Unit and integration test source files
----------------	--

src/test/resources/	Resources used only during testing
---------------------	------------------------------------

target/	All generated output — compiled classes, JAR files, test reports
---------	--

pom.xml	The project's Maven configuration file
---------	--

Why this structure matters:

- Maven automatically compiles everything in src/main/java/
 - Maven automatically runs test classes in src/test/java/
 - Resources in src/main/resources/ are automatically included in the final JAR
 - The target/ directory is always safe to delete — mvn clean removes it and it can be fully regenerated
-